HAMK
Hämeen ammatti-
korkeakoulu

DevOpsin ymmärtäminen: käytännöt, periaatteet ja hyödyt

Tietojenkäsittely

Kevät 2026

Jaakko Kulmala

Sisällys

1 DevOps ohjelmistokehityksessä 1

2 DevOps: Kulttuurista käytäntöön ja tuloksiin 1

2.1 Periaatteet ja hyödyt 1

2.2 Mittaukset ja hyödyt 2

3 Työkalut ja teknologiat 2

4 DevOpsin haasteet 3

5 Netflix case-study 3

6 Yhteenveto 4

Lähteet 5

1 DevOps ohjelmistokehityksessä

DevOps tarkoittaa ohjelmistokehityksen toimintatapaa, jossa kehitys- ja ylläpitotiimit työskentelevät tiiviisti yhdessä koko ohjelmiston elinkaaren ajan. Sen tavoitteena on nopeuttaa ohjelmistojen kehitystä, testausta, käyttöönottoa ja ylläpitoa sekä parantaa laatua ja luotettavuutta. DevOpsin rooli on ohjelmistokehityksen prosessissa automatisoida ja tehostaa työnkulkua, kuten jatkuvaa integraatiota ja jatkuvaa toimitusta, jotta ohjelmistoja voidaan julkaista nopeammin, turvallisemmin ja vähemmillä virheillä. Panostamalla yhteistyöhön yritys voi kasvaa sekä ammatillisesti että toiminnallisesti, kun työresursseja käytetään oikein. (Microsoft, n.d.)

Perinteiseen ohjelmistokehityksen lähestymistapaan kehitys- ja ylläpitotiimien työskentely erillään toisistaan, terminologina käytetään sanaa siilo. Erona DevOpsiin, nämä siilot estävät usein tiimien välisen sujuvan yhteistyön. Ylläpitotiimillä ei ole suoraa tietoa kehitystiimin edistymisestä hankaloittaen monitorointisuunnitelman kehittämistä. (Ugochukwu, 2024)

2 DevOps: Kulttuurista käytäntöön ja tuloksiin

DevOps ei ole pelkkä tekninen työkalupakki, vaan kokonaisvaltainen organisaatiotason sitoutuminen, jossa kulttuuri, koulutus ja mentorointi ovat onnistumisen edellytyksiä (Nieminen, 2024). Sen ytimessä on jaettu vastuu ja avoin kommunikaatio, jotka murtavat perinteiset siilot kehitys- ja operatiivisten tiimien väliltä (Atlassia, n.d.). Ilman vahvaa luottamuksen ja jatkuvan oppimisen kulttuuria DevOps on vain pelkistetty tekninen prosessi, kykenemätön uudistamaan tiimien todellisia toimintatapoja tai saavuttamaan sille asetettuja laadullisia tavoitteita.

2.1 Periaatteet ja hyödyt

Tekninen suorituskyky saavutetaan hyödyntämällä CI/CD-putkea, Continuous Integration/Delivery, joka automatisoi koodimuutosten testauksen ja julkaisun, minimoiden inhimilliset virheet (Red Hat, 2025). Tätä tukee infrastruktuurin hallinta koodina, IaC, jolloin verkkoja ja palvelimia operoidaan ohjelmallisesti manuaalisen konfiguroinnin sijaan, sekä jatkuva seuranta ja kirjaaminen, jotka mahdollistavat poikkeamien havaitsemisen reaaliajassa (Microsoft, 2025). Nämä käytännöt luovat perustan nopealle ja ennustettavalle ohjelmistotuotannolle, jossa muutokset viedään tuotantoon pienissä, hallituissa ja turvallisissa erissä.

Toiminnan jatkuva kehittäminen vaatii siirtymistä syyttelyn kulttuurista systeemiseen analyysiin, jota edustaa esimerkiksi Googlen SRE-malli, Site Reliability Engineering, ja sen ”blameless postmortem” -käytäntö: virheen sattuessa keskitytään yksilön sijaan järjestelmän heikkouksiin (Lueder & Lunney, n.d.). Jatkuva palautejärjestelmä, joka hyödyntää mittareita kuten läpimeno- ja palautusaikaa, mahdollistaa ketterän reagoinnin asiakastarpeisiin ja teknisiin haasteisiin. Tämä periaatteellinen taso varmistaa, että automaatio ja prosessit palvelevat todellista liiketoimintatavoitetta ja parantavat tiimin suorituskykyä systemaattisesti.

2.2 Mittaukset ja hyödyt

Tulokset ovat mitattavissa suoraan nopeudessa, laadussa ja tehokkuudessa, mikä parantaa sekä asiakaskokemusta että työntekijöiden hyvinvointia (Salesforce, n.d.). Käytännön esimerkit, kuten Capital One, joka puolitti koodin läpimenoajan, tai City of Hope, joka tehosti kehittäjien tuottavuutta 25 % ja pudotti kasausajat murto-osaan, osoittavat DevOpsin konkreettisen voiman (KodeKloud, 2024). Kun automaatio poistaa rutiininomaisen manuaalisen työn, organisaatio voi keskittyä arvon tuottamiseen ja reagoida markkinoiden muutoksiin nopeammin ja luotettavammin.

3 Työkalut ja teknologiat

DevOps luottaa moniin työkaluihin ja teknologioihin, jotka tukevat yhteistyötä, automaatiota, käyttöönottoa ja seurantaa. Yksi monipuolisimmista työkaluista on Microsoftin tarjoamat Git, Github, GitLab ja Azure DevOps -versionhallinta järjestelmät, jotka soveltuvat täydellisesti yhteistyöhön. Jatkuvan integraation ja toimituksen, CI, toimii puolestaan edellä Git -tuoteperheen lisäksi Jenkins ja konttiliikenteeseen Docker ja Kubernetes. Terraform ja Ansible kuuluvat IaC ja konfiguraationhallintaan, Splunk on erittäin vahva valinta seurantaan ja lokien tallennukseen.

Näiden työkalujen avulla voidaan hallita koodia, automatisoida työnkulkuja, ottaa ohjelmistoa käyttöön nopeammin ja valvoa sovellusten suorituskykyä tehokkaasti. (Random Articles, 2025)

DevOps-työkalujen tärkein toiminta on organisaatioiden skaalautuvuuden ohjelmistotoimitusten suhteen tehostamiseksi. Automaatiotyökalut vähentävät manuaalista työtä testauksessa, rakentamisessa ja käyttöönotossa. Seurannassa työkalut auttavat havaitsemaan ongelmat

reaaliajassa ja tukevat nopeampaa reagointia. Hallinnan työkalut parantavat näkyvyyttä, versionhallintaa ja yhdenmukaisuutta eri ympäristöissä.

Esimerkiksi finanssiteknologia-alan startup yritys kohtasi kasvaessaan haasteita laajentua, kun asiakaskunta kasvoi. Dockerilla he saivat vakaamman ympäristön kehitykseen, testaukseen ja tuotantoon. Kuberneteksella he saivat kontit laajentumaan automaattisesti edistäen resurssien kohdennusta haasteellisempiin tehtäviin. Jenkinsin avulla automaatiotestausputki saatiin toimimaan edistäen uusien ominaisuuksien ja päivitysten lähetystä sekä nopeuttaa ja vakauttaa tuotantoa. (Kaleynik, 2024)

4 DevOpsin haasteet

Muutosvastarinta on yritystasolla yksi suurimmista haasteista, jota DevOps kulttuurin tuonnissa saattaa tulla vastaan. Mikäli työntekijät ovat tottuneet työskentelemään siiloissa, yhteistyötä voi olla vaikea parantaa pakottamalla heitä uudistumaan. Tiimit saattavat myös kohdata ongelmia uusien työkalujen kanssa, jos tarkoituksena olisi integroida vanhojen järjestelmien kanssa, sillä ne eivät ole suunniteltu nykyaikaisille DevOps-menetelmille.

5 Netflix case-study

Netflixin DevOps-menestyksen taustalla oli usean tärkeän tekijän yhdistelmä, joista keskeisin oli pilvipohjaiseen infrastruktuuriin siirtyminen, AWS:ään. Tämä mahdollisti palvelun nopean skaalautumisen maailmanlaajuisesti käyttäjämäärien kasvaessa nopeasti, jolloin järjestelmän on kestää suuria kuormituksia.

Sen sijaan, että koko palvelu olisi ollut yksi suuri järjestelmä, Netflix jakoi sen pienempiin itsenäisiin palveluihin. Tämän mikropalveluarkkitehtuurin avulla kehitys helpottui, julkaisuja saatiin nopeammin eteenpäin ja vähensi riskiä, että yhden osan virhe kaataisi koko palvelun.

Ohjelmiston testausta, käyttöönottoa ja ylläpitoa automatisoimalla vähentyi manuaalisten virheiden määrä ja uusien ominaisuuksien julkaisu nopeutui. Näin Netflix hyödynsi CI/CD kehitysputkea. Tärkeimpänä onnistumisen syynä oli myös kulttuuri. Tiimeillä oli paljon vastuuta omista palveluistaan ja yhteistyö kehityksen ja operaatioiden välillä oli vahvaa. Tämä vastaa hyvin DevOpsin ydintä eli nopeaa kehitystä, jatkuvaa parantamista, yhteistyötä ja järjestelmien luotettavuutta.

6 Yhteenveto

DevOps on tärkeä moderni lähestymistapa ohjelmistokehitykseen, jossa yhdistyvät ihmiset, prosessit ja työkalut ohjelmistotoimitusten parantamiseksi. Sen käytännöt ja periaatteet auttavat tiimejä työskentelemään nopeammin, tehokkaammin ja laadukkaammin. Yrityksen on turha yrittää siirtyä DevOps-kulttuuriin vain ostamalla kalliita työkaluja, kuten Jiraa tai pilvipalveluita muuttamatta ihmisten tapaa kommunikoida, sillä tällöin vain tuhlataan rahaa. Aloittamalla pienestä yksittäisestä projektista, esimerkiksi automatisoimalla sen testaus ja poistamalla manuaaliset välivaiheet ja tarkkailemalla oikeita mittareita, kuten MTTR eli Mean Time to Repair/Recovery. Osallistuttamalla molempia tiimejä katsomaan monitorointidataa sekä osallistumaan koodikatselmointiin saadaan tiimien välisiä siiloja hajotettua, joka parantaa yhteistyötä sekä mahdollisuutta omaksua DevOps-kulttuuri.

Lähteet

Microsoft. (n.d.). What is DevOps?
https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-devops/

Ugochukwu, M. (2024). DevOps vs. Traditional: Key Differences in Software Development and Delivery.
https://dev.to/princemaxi/devops-vs-traditional-key-differences-in-software-development-and-delivery-2j0g

Nieminen, J. (2024). DevOps-kulttuurin vaikutus ohjelmistokehityksen tehokkuuteen.
https://turvataito.fi/devops-kulttuurin-vaikutus-ohjelmistokehityksen-tehokkuuteen/

Red Hat. (2025). What is a CI/CD pipeline?
https://www.redhat.com/en/topics/devops/what-cicd-pipeline#what-is-a-ci/cd-pipeline

KodeKloud. (18.12.2024). 4 Companies That Prove DevOps Is the Future | DevOps Case Studies | KodeKloud [video]. Youtube. https://www.youtube.com/watch?v=KJI_hK0tohY

5 Key DevOps principles. (n.d.). *Atlassian.* https://www.atlassian.com/devops/what-is-devops

Lueder, S., Lunney, J. (n.d.). Postmortem Culture: Learning from Failure.
https://sre.google/sre-book/postmortem-culture/

9 Benefits Of DevOps For Your Business. (n.d.). Salesforce.
https://www.salesforce.com/platform/devops-tools/what-is-devops/benefits/

Random Articles. (2025). DevOps Tools and Technologies: A Practical Guide for Modern Teams.
https://medium.com/@yogxworld/devops-tools-and-technologies-a-practical-guide-for-modern-teams-e5b83f2ec294

Egor Kaleynik. (2024). Case studies: Devops success stories of startups.
https://vallettasoftware.com/pillars/devops-success-stories